

GROUP TECH BRIEF / 2026.08.25

从 Codex 到个人 Agent

Tibo × Matthew Berman 访谈要点与 AI Infra 解读

组内分享版 · 阅读时间约 8 分钟 · 原视频约 44 分钟

一句话结论

这场访谈真正讨论的不是“更强的代码补全”，而是把 Codex 的执行能力、ChatGPT 的统一入口和云端算力合成一个长期运行的个人 Agent。对 AI Infra 而言，竞争指标也会从单算子吞吐继续上移到端到端完成时间。

四个最值得关注的信号

01 产品形态 ChatGPT 与 Codex 的边界逐渐消失，用户只面对一个统一 Agent。	02 交互方式 超高速推理把“异步派活”重新拉回实时协作与连续反馈。
03 基础设施 笔记本更像控制台，真正的并行执行、沙箱和状态管理迁移到云端。	04 研发飞轮 模型开始参与 Kernel、训练系统、Serving 与 Agent Harness 的优化。

说明：下文将“访谈明确观点”“技术解读”“外部资料佐证”分开呈现。

OVERVIEW

访谈背景与章节地图

先确认受访者、主题边界与阅读方式

受访者：Thibault “Tibo” Sottiaux，在 OpenAI 负责 Codex；OpenAI Forum 资料显示，他此前在 Google DeepMind 负责 Gemini 人类数据工作并建设 AI/ML 工作流基础设施。[2]

访谈定位：这是一场友好型人物访谈，适合观察产品与组织方向，但不应被视为正式技术白皮书或确定路线图。

章节地图（按原视频时间戳）

时间	主题	时间	主题
0:45	在 Google DeepMind 的经验	4:22	OpenAI 的高执行力文化
7:23	Astra 与下一代模型	11:18	速度如何改变开发工作流
14:27	ChatGPT 与 Codex 合流	20:25	OpenAI 与 Anthropic
23:37	为什么持续重置额度	26:41	AI 效率与计算资源
30:25	递归自我改进	32:00	Frontier AI “暂停” 与风险
34:13	Ultrafast 会成为默认吗	43:20	为什么每个人都应尝试 AI

阅读标记

访谈观点	Tibo 在访谈中表达的个人判断；代表一手观察，但不等于公司正式承诺。
技术解读	结合 AI Infra、Serving 与 Agent Runtime 做出的系统推演；属于分析，不是访谈原话。
外部佐证	OpenAI 官方资料或公开报道，用来核对速度指标、身份背景和产品方向。

最重要的阅读原则 不要把“方向判断”直接等同于“已经落地”。本报告更关注趋势背后的技术结构，而不是追逐单个产品名词。

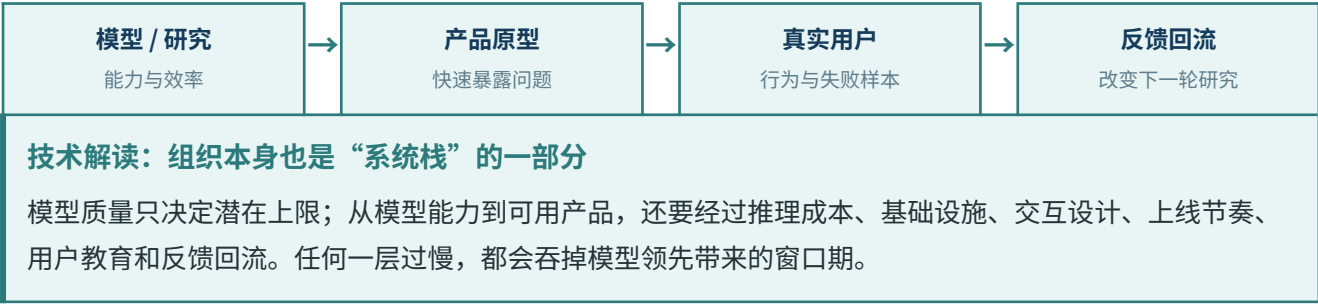
组织与产品化：模型能力不等于产品领先

Tibo 对 DeepMind 与 OpenAI 的比较，更像在讨论 “反馈回路速度”

访谈观点：DeepMind 在 ChatGPT 发布前已经拥有相似能力与内部原型，但研究导向、组织链路与发布节奏，使这些能力没有及时变成大规模消费产品；OpenAI 的特点则是小团队、自主决策、快速上线和强用户反馈。

研究机构式优势	产品-研究一体化优势
模型、研究和基础设施能力强	研究与产品团队距离更短
更倾向在内部继续完善	先发布，再用真实反馈快速修正
发布决策链可能更长	小团队可以快速形成闭环
风险是 “有能力但没形成入口”	风险是快速迭代带来的稳定性与治理压力

真正的竞争闭环



需要打折：这是 Tibo 的个人经历与组织观察，不是对 Google/DeepMind 全部历史的完整复盘。

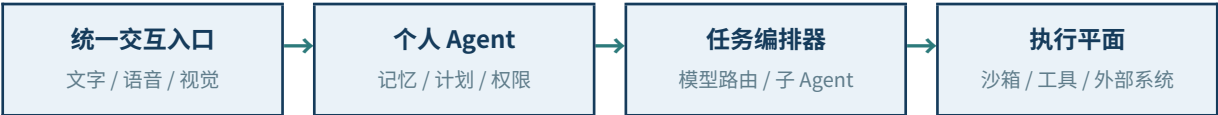
Codex → 通用个人 Agent

未来用户管理的不是十几个 Agent，而是一个理解自己、能调用所有工具的统一入口

为什么说今天的 Coding Agent 仍然“原始”

- 高级用户仍需手动维护规则文件、Skills、Memory、MCP、权限和环境。
- 多 Agent 并行时，用户实际上同时承担任务拆分、调度、验收和故障处理。
- 终端、IDE、浏览器、云任务之间来回切换，破坏连续的协作体验。

目标形态：一个长期存在的个人 Agent



访谈方向与 WIRED 对 ChatGPT/Codex 整合的报道相互印证：底层可由 Codex 写代码、调用 API 或浏览网页，但普通用户只需要自然语言表达目标。 [6]

为什么笔记本会成为瓶颈

本地：控制平面	云端：执行平面
低延迟交互、隐私边界、用户确认	大量并行任务、弹性算力、长任务运行
展示状态、结果与中断入口	隔离沙箱、依赖安装、编译测试、浏览器自动化
保留少量本地工具和缓存	状态持久化、失败重试、凭证隔离、观测与审计

AI Infra 推论

未来 Agent 平台更像“面向任务的云操作系统”：核心不只是模型 API，还包括调度、隔离、权限、状态、恢复、工具成功率和尾延迟。

Ultrafast：速度会改变工作流，而不只是“体感更快”

当模型能够跟上人的注意力，交互会从异步委派重新回到实时协作

官方公开指标	数值	应如何理解
GPT-5.6 Sol Ultrafast	最高 14×；最高 750 output tok/s	“up to”，且为有限预览，不代表所有请求平均值。[3]
GPT-5.3 Codex-Spark	超过 1000 tok/s	针对实时编程的小模型与低延迟硬件路径。[4]
端到端 Harness 优化	往返开销 -80%；每 token 开销 -30%；TTFT -50%	说明模型采样速度只是总延迟的一部分。[4]

交互模式的变化

较慢的 Agent：异步派活	超快的 Agent：实时结对
一次给出大任务，等待较长时间	以小步骤连续提出、观察、纠偏
用户同时开多个任务，承担调度压力	用户保持在同一注意力窗口内
适合大规模重构、批处理、长研究	适合调试、交互式实验、UI 迭代、语音
重点是吞吐与独立完成能力	重点是 TTFT、ITL、可中断和连续反馈

为什么 14× 模型速度，端到端可能只有约 3.9×

Amdahl 示例 原任务：80 s 模型 + 20 s 其他 = 100 s 加速后：80/14 + 20 ≈ 25.7 s 端到端加速：100/25.7 ≈ 3.9×	当模型生成占总时延 80% 时，即使生成阶段提升 14×，工具、网络、沙箱和测试仍然不会自动加速。模型越快，瓶颈越快迁移到剩余链路。
--	---

端到端延迟拆解：T_total = T_queue + T_prefill + T_decode + T_network + T_tool + T_sandbox + T_verify

优化后的新瓶颈

请求排队、Prefill/TTFT、Tool RPC、沙箱冷启动、Git checkout、依赖初始化、编译测试、多 Agent 同步，以及 P95/P99 长尾。

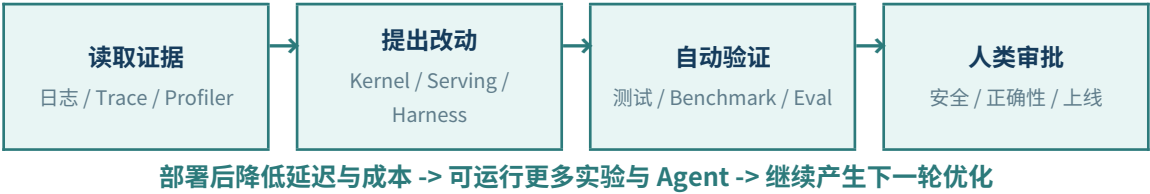
递归自我改进：更准确地理解为“工程系统飞轮”

不是模型偷偷改权重，而是模型参与优化承载自己的整套系统

先排除一个误解

访谈中的 Recursive Self-Improvement 不应被直接理解为“模型完全自主训练下一代模型并无限升级”。更现实的早期形态，是有人类审批、评测和部署约束的 AI-assisted system co-optimization。

工程闭环



模型可以参与哪些 AI Infra 工作

方向	典型任务
Kernel / Compiler	读 profiler、定位 bound、生成与调优 CUDA Kernel、检查数值正确性
训练系统	诊断失败、优化训练配方、分析通信与数据管线、自动跑实验
推理与 Serving	分析 TTFT/TPOT、批处理与队列、KV Cache、路由和尾延迟
Agent Harness	改进 Tool Calling、重试策略、上下文管理、沙箱与任务恢复
评测与回归	生成测试、自动比较版本、发现性能或正确性退化

外部佐证：OpenAI 在 GPT-5.6 发布资料中称，内部研究者已使用模型诊断失败、优化训练系统、运行实验，并用包括研究调试、Kernel 生成和模型改进在内的评测衡量 RSI 能力。[5]

必须保留的边界

Eval 通过不等于可以无人监管上线。真正可靠的闭环仍需要基准稳定性、独立验证、权限隔离、回滚机制、人工审批和安全监控。

对组内 AI Infra 工作的直接启发

从“把某个 Kernel 做快”升级到“把用户任务的完成闭环做快”

性能指标应分层观察

层级	核心指标	常见瓶颈	优化后会迁移到哪里
Kernel	FLOPS、带宽、Occupancy、Launch	Compute / Memory / 同步	通信、Launch、上层调度
模型	TTFT、TPOT、KV、MoE imbalance	Prefill、Decode、通信	队列、工具、外部依赖
Serving	吞吐、排队、P95/P99、成本	Batch、Admission、资源碎片	沙箱与 Tool RPC
Agent Runtime	工具成功率、冷启动、重试、恢复	编排、权限、外部系统	人的确认与注意力
用户任务	Time-to-useful-result、成功率、纠偏轮数	全链路耦合	业务流程与组织决策

值得优先补齐的六项能力

- 1. 端到端 Trace：把模型、网络、工具、沙箱、编译测试放进一条可关联的时间线。
- 2. 区分吞吐与交互延迟：离线批处理关注 throughput，实时 Agent 更关心 TTFT、ITL、可中断性。
- 3. 重视尾延迟：P50 很漂亮但 P99 被 Tool RPC 或冷启动拖垮，用户仍会觉得系统不可靠。
- 4. 做流水重叠：让 Prefill/Decode、Tool Call、数据搬运、验证和后续任务尽量并行。
- 5. 速度-能力-成本路由：实时小任务用快模型，复杂长任务用强模型，后台可并行子 Agent。
- 6. AI 辅助系统优化：让模型参与 Kernel、Profiler、日志和回归分析，但用稳定 Eval 与人工审核闭环。

项目复盘时建议固定问四个问题

① 我们改善了 T_total 的哪一项？② 改善的是 P50 还是 P95/P99？③ 原瓶颈消失后，新瓶颈在哪里？④ 真实任务的完成时间和成功率是否同步改善？

需要打折看的地方与参考资料

把方向、事实和推演分开，才能避免被访谈气氛与“up to”数字带偏

五个保留项

1. **组织比较带有个人视角**：Tibo 的经历很有价值，但不能覆盖 DeepMind、Google 和 OpenAI 的全部决策背景。
2. **Tokens/s 不等于端到端体验**：14×、750 tok/s 和 1000+ tok/s 都是特定模式或硬件路径下的公开上限。
3. **产品合流是方向，不是固定形态**：ChatGPT 与 Codex 的界面、权限、商业模式和时间表仍可能变化。
4. **云端 Agent 架构属于技术推演**：访谈指出单机资源会成为瓶颈，但本文给出的控制平面/执行平面是工程化解释。
5. **“暂停”与 RSI 细节仍不充分**：访谈没有公开完整触发事件、评测阈值和恢复标准，不宜过度推断。

最终结论

AI Infra 的价值正在上移

Kernel 优化仍然重要，但更高价值的问题是：这部分加速在模型、Serving、Agent Runtime 和真实用户任务中还能剩下多少？谁能持续找到“优化后的下一瓶颈”，谁就更接近下一代 Agent 基础设施。

参考资料

- [1] 原始 X 帖与完整访谈：[链接 1](#) / [链接 2](#)
- [2] OpenAI Forum：Tibo 背景与 Codex 职责：[链接 1](#)
- [3] OpenAI：Previewing Ultrafast mode：[链接 1](#)
- [4] OpenAI：Introducing GPT-5.3-Codex-Spark：[链接 1](#)
- [5] OpenAI：GPT-5.6 与内部 AI 研发加速 / RSI 评测：[链接 1](#)
- [6] WIRED：ChatGPT 与 Codex 的核心产品整合报道：[链接 1](#)

本报告基于公开访谈和截至 2026-08-25 的公开资料整理，仅供内部讨论。